

SON algorithm

Purpose of SON Algorithm

The SON Algorithm is a variant of the Apriori Algorithm used for frequent itemset mining on large datasets, especially in distributed systems like MapReduce. Instead of processing the entire dataset in one go, it processes it in smaller chunks or partitions to make the mining of frequent patterns more efficient and scalable.

How SON Algorithm Works:

1. **Divide the dataset** into partitions.
2. **Find local frequent itemsets** from each partition using a local support threshold.
3. **Generate candidate frequent itemsets** by combining local frequent itemsets.
4. **Validate candidate itemsets** against the entire dataset to identify globally frequent itemsets.

Example:

Dataset (Transactions):

Imagine you have a small transactional dataset with 6 transactions, where each transaction contains a set of items (e.g., products purchased together).

Transaction ID	Items
T1	{A, B, C}
T2	{A, B}
T3	{A, C}
T4	{B, C}
T5	{A, C, D}
T6	{B, C, D}

- The dataset has 6 transactions with items labeled A, B, C, and D.
- We want to find **frequent itemsets**, meaning sets of items that appear together frequently in the transactions.

Step 1: Partition the dataset

Let's divide the dataset into two **partitions** to make it easier to handle.

- **Partition 1:** {T1, T2, T3}
 - T1: {A, B, C}
 - T2: {A, B}
 - T3: {A, C}
- **Partition 2:** {T4, T5, T6}
 - T4: {B, C}
 - T5: {A, C, D}
 - T6: {B, C, D}

Step 2: Pass 1 - Find Local Frequent Itemsets

Now, we need to find **frequent itemsets** in each partition **locally**.

- Let's say the **global minimum support** threshold (the number of times an itemset must appear to be considered frequent) is 2.
- Since we have divided the dataset into 2 partitions, we adjust the **local support threshold** to $\text{min_support} / \text{number_of_partitions} = 2 / 2 = 1$.
 - So, in each partition, an itemset must appear **at least once** to be considered frequent.
- **Partition 1 (T1, T2, T3):**
 - Count the occurrences of itemsets:
 - **1-itemsets:**
 - A: 3 times (in T1, T2, T3)
 - B: 2 times (in T1, T2)
 - C: 2 times (in T1, T3)
 - **2-itemsets:**
 - {A, B}: 2 times (in T1, T2)
 - {A, C}: 2 times (in T1, T3)
 - {B, C}: 1 time (in T1)
 - **3-itemsets:**
 - {A, B, C}: 1 time (in T1)

Frequent itemsets in Partition 1:

- **1-itemsets:** {A}, {B}, {C}
- **2-itemsets:** {A, B}, {A, C}
- **3-itemsets:** None (because {A, B, C} appears only once, which is below the threshold).
- **Partition 2 (T4, T5, T6):**
 - Count the occurrences of itemsets:
 - **1-itemsets:**
 - A: 1 time (in T5)
 - B: 2 times (in T4, T6)
 - C: 3 times (in T4, T5, T6)
 - D: 2 times (in T5, T6)
 - **2-itemsets:**
 - {B, C}: 2 times (in T4, T6)
 - {A, C}: 1 time (in T5)
 - {B, D}: 1 time (in T6)
 - {C, D}: 2 times (in T5, T6)
 - **3-itemsets:**
 - {B, C, D}: 1 time (in T6)

Frequent itemsets in Partition 2:

- **1-itemsets:** {B}, {C}, {D}
- **2-itemsets:** {B, C}, {C, D}
- **3-itemsets:** None (because {B, C, D} appears only once, which is below the threshold).

Step 3: Combine Local Frequent Itemsets

Now, we combine the frequent itemsets from both partitions to generate **candidate frequent itemsets**:

- **Frequent 1-itemsets:** {A}, {B}, {C}, {D}
- **Frequent 2-itemsets:** {A, B}, {A, C}, {B, C}, {C, D}

Step 4: Pass 2 - Global Validation

In this pass, we go through the entire dataset (all 6 transactions) to count how many times the **candidate frequent itemsets** actually appear.

- Count occurrences of the candidate frequent itemsets in the entire dataset:
 - {A, B} → 2 times (in T1, T2)
 - {A, C} → 3 times (in T1, T3, T5)
 - {B, C} → 3 times (in T1, T4, T6)
 - {C, D} → 2 times (in T5, T6)

Step 5: Identify Global Frequent Itemsets

Now, based on the global support threshold (which is 2), the following itemsets are considered globally frequent:

- **Frequent 2-itemsets:** {A, B}, {A, C}, {B, C}, {C, D}

Final Result

The frequent itemsets in the dataset are:

- {A, B}
- {A, C}
- {B, C}
- {C, D}

CURE Algorithm

Steps of the CURE Algorithm:

1. Random Sampling:

- To reduce computational complexity, CURE first selects a **random sample** from the entire dataset. This subset of points represents the full dataset while keeping the processing manageable.
- The assumption is that a small random sample is a good representation of the overall dataset. The random sample will be clustered instead of the entire dataset, significantly reducing the computational burden.

2. Initial Clustering Using Hierarchical Approach:

- Once the sample is selected, an **agglomerative hierarchical clustering** algorithm is applied to form initial clusters.
- In hierarchical clustering, every data point starts as its own cluster, and then the two nearest clusters are repeatedly merged until the required number of clusters remains.
- Various linkage methods can be used to determine the distance between clusters, such as:
 - **Single-link**: Distance between the closest pair of points in different clusters.
 - **Complete-link**: Distance between the farthest pair of points in different clusters.

3. Selecting Representative Points:

- For each cluster, a set of **representative points** is chosen. These points are spread out within the cluster to capture its shape and structure.
- The number of representative points per cluster is pre-defined (let's say, k points are selected from each cluster).
- This step ensures that the clusters are represented by more than just a single centroid. Instead, multiple points capture the diversity of the cluster.

4. Shrinking Representative Points:

- The representative points chosen for each cluster are **shrunk** towards the cluster's centroid by a fixed fraction, α (typically 0.2 or 20%).
- Shrinking helps to reduce the effect of outliers because points that are farther from the cluster's center are brought closer to the core, making the cluster tighter and more compact.
- The shrinking factor can be adjusted based on the dataset characteristics and how tight or loose the clusters should be.

5. Merging Clusters:

- After choosing and shrinking the representative points, the clusters are merged iteratively.
- The **distance between clusters** is calculated based on the distance between their representative points. Typically, the **minimum distance** between representative points of two clusters is used to decide which clusters to merge.
- This merging continues until the desired number of clusters is achieved.

6. Handling Larger Datasets:

- For very large datasets, CURE uses **partitioning**. The dataset is divided into smaller parts, and CURE is applied to each partition independently. After processing each partition, the partial results are combined and the final clusters are formed using the representative points from all partitions.

aExample of CURE Algorithm:

Let's go through a more detailed example to understand how the CURE algorithm works in practice.

Dataset:

Consider the following 12 data points in a 2D space:

Point Coordinates

P1	(2, 3)
P2	(3, 3)
P3	(4, 4)
P4	(5, 3)
P5	(9, 10)
P6	(10, 9)
P7	(9, 9)
P8	(15, 15)
P9	(16, 16)
P10	(30, 30)
P11	(31, 31)
P12	(32, 30)

We can see there are roughly four clusters in this dataset:

- Cluster 1: {P1, P2, P3, P4}
- Cluster 2: {P5, P6, P7}
- Cluster 3: {P8, P9}
- Cluster 4: {P10, P11, P12}

Step-by-Step Application of CURE:

Step 1: Random Sampling

- In this small example, assume we use all 12 points (no sampling needed due to small dataset).

Step 2: Hierarchical Clustering

- Start by treating each point as its own cluster and merge the closest ones.
 - Merge {P1, P2}, {P5, P6}, {P10, P11} (small distances).
 - Continue merging until we have four clusters left:
 - Cluster 1: {P1, P2, P3, P4}
 - Cluster 2: {P5, P6, P7}
 - Cluster 3: {P8, P9}
 - Cluster 4: {P10, P11, P12}

Step 3: Selecting Representative Points

- Select **two representative points** from each cluster (this is customizable based on the problem).
 - Cluster 1: Representative points = {P1, P4}
 - Cluster 2: Representative points = {P5, P7}
 - Cluster 3: Representative points = {P8, P9}
 - Cluster 4: Representative points = {P10, P12}

Step 4: Shrinking Representative Points

- Shrink the representative points towards the centroid of their respective clusters (e.g., shrink by 20%).
 - For example, in Cluster 1:
 - Centroid of {P1, P4} = (3.5, 3)
 - Shrink P1 (2, 3) towards the centroid:
 - New position of P1 = $(2 + 0.2 \times (3.5 - 2), 3 + 0.2 \times (3 - 3)) = (2.3, 3)$
 - Shrink P4 (5, 3) towards the centroid:
 - New position of P4 = $(5 + 0.2 \times (3.5 - 5), 3 + 0.2 \times (3 - 3)) = (4.7, 3)$

Step 5: Merging Clusters

- Calculate the distance between the closest pair of representative points between clusters:
 - For instance, if the distance between the representative points of Cluster 1 and Cluster 2 is small, merge them.
 - This process continues until the desired number of clusters is reached (e.g., two final clusters).

Example of Canopy Clustering:

Key Concepts:

1. **Cheap Distance Metric (Loose Bound):** Canopy clustering uses a simple, computationally cheap distance metric to determine rough groupings of data points. For example, Manhattan distance or Euclidean distance with some approximations.
2. **Two Thresholds (T1 and T2):** Canopy clustering works with two distance thresholds:
 - **T1 (Loose Threshold):** This is a larger distance that determines whether a data point should be considered as part of a "canopy."
 - **T2 (Tight Threshold):** This is a smaller distance used to refine the canopy by determining stronger membership within it.
3. **Canopy:** A **canopy** is a cluster or group of data points that are close to each other based on the loose threshold (T1). Points within the tight threshold (T2) are considered strongly associated with the canopy, while points only within the loose threshold (T1) are more loosely associated.

Steps of Canopy Clustering:

1. **Start with the Dataset:** Assume we have a dataset of points and we need to form clusters.
2. **Select a Point Randomly:** Choose a point randomly from the dataset as a "canopy center."
3. **Calculate Distance:**
 - Calculate the distance between the selected point and all other points in the dataset using the cheap distance metric.
 - If the distance to a point is less than **T1**, add that point to the current canopy.
 - If the distance is also less than **T2**, mark it for stronger association with the canopy.
4. **Remove Points Covered by the Canopy:** Once a canopy is formed, remove the points that are strongly associated (i.e., within distance **T2**) from the dataset to avoid redundant clustering.
5. **Repeat Until Dataset is Empty:** Repeat steps 2-4 until no points are left unclustered.
6. **Refinement (Optional):** After forming canopies, apply a more sophisticated clustering algorithm (e.g., k-means) to the points within each canopy to further refine the clustering.

Consider a set of 10 2D data points:

Point	Coordinates
P1	(1, 1)
P2	(2, 1)
P3	(1, 2)
P4	(10, 10)
P5	(10, 11)
P6	(11, 10)
P7	(50, 50)
P8	(51, 50)
P9	(50, 51)
P10	(100, 100)

Let's define our thresholds:

- **T1** (loose threshold) = 15
- **T2** (tight threshold) = 5

Step-by-Step Process:

Step 1: Start with Random Point Selection

- Randomly pick a point, say **P1 (1, 1)**.
- Calculate the distance between **P1** and all other points in the dataset using a cheap distance metric, like Euclidean distance.

Step 2: Form a Canopy Based on Thresholds

- Calculate distances:
 - Distance(P1, P2) = 1
 - Distance(P1, P3) = 1
 - Distance(P1, P4) $\approx$ 12.73
 - Distance(P1, P5) $\approx$ 13.45
 - Distance(P1, P6) $\approx$ 14.21
 - Distance(P1, P7) $\approx$ 69.30
 - Distance(P1, P8) $\approx$ 70.01
 - Distance(P1, P9) $\approx$ 70.01
 - Distance(P1, P10) $\approx$ 140.01
 - Points **P2 (2, 1)** and **P3 (1, 2)** are within the tight threshold **T2 = 5**, so they are strongly associated with the canopy.
 - Points **P4 (10, 10)**, **P5 (10, 11)**, and **P6 (11, 10)** are within the loose threshold **T1 = 15**, so they are part of the canopy but more loosely associated.

So, the canopy formed by **P1** contains {P1, P2, P3, P4, P5, P6}.

Step 3: Remove Points Covered by the Canopy

- Now, remove the points {P2, P3} from the dataset since they are strongly associated with **P1's** canopy.

Remaining dataset: {P4, P5, P6, P7, P8, P9, P10}

Step 4: Repeat the Process

- Select the next random point, say **P4 (10, 10)**.
- Form a new canopy based on **P4**:
 - Distance(P4, P5) = 1
 - Distance(P4, P6) = 1
 - Distance(P4, P7) ≈ 56.57
 - Distance(P4, P8) ≈ 57.28
 - Distance(P4, P9) ≈ 57.28
 - Distance(P4, P10) ≈ 127.28
 - Points **P5 (10, 11)** and **P6 (11, 10)** are strongly associated with **P4** (within **T2**).
 - No other points are within the loose threshold **T1**.

Canopy formed: {P4, P5, P6}.

- Remove {P5, P6} from the dataset.

Remaining dataset: {P7, P8, P9, P10}

Step 5: Repeat Again

- Pick **P7 (50, 50)** and form a canopy.
 - Points **P8 (51, 50)** and **P9 (50, 51)** are within **T2**, and they form a strong association with **P7**.

Canopy formed: {P7, P8, P9}.

- Remove {P8, P9} from the dataset.

Remaining dataset: {P10}

Step 6: Form Final Canopy

- Pick the remaining point **P10 (100, 100)**. Since it's the only point left, it forms a canopy by itself.

Final canopy: {P10}.

Summary of Canopies:

- Canopy 1: {P1, P2, P3, P4, P5, P6}
- Canopy 2: {P7, P8, P9}

- Canopy 3: {P10}

After this, a more refined clustering algorithm like k-means can be applied to each canopy to fine-tune the clustering.

Advantages of Canopy Clustering:

1. **Scalability:** The algorithm is very fast, making it suitable for clustering large datasets.
2. **Dimensionality Reduction:** By forming rough groups (canopies), it reduces the number of distance computations for more expensive algorithms like k-means.
3. **Easy Implementation:** Canopy clustering is simple to implement and understand.

Disadvantages:

1. **Approximate Clustering:** It is a rough, approximate clustering method. The final clustering may need further refinement through another algorithm.
2. **Choice of Thresholds:** The success of the algorithm depends heavily on the choice of **T1** and **T2**. Poorly chosen thresholds can lead to overlapping canopies or ineffective clustering.

Canopy Clustering is a quick and efficient way to preprocess large datasets, reducing the computational load for more precise clustering algorithms. By using two distance thresholds (T1 and T2), it forms rough clusters (canopies) and allows fine-grained clustering to be applied later. This approach is particularly effective when handling large datasets, offering scalability and simplicity at the cost of some accuracy.